

Documento Técnico de Elaboración de la Solución

**Framework de Calidad, Automatización y Migración de Datos
Biblioteca Central**

Realizado por: Daniela Quinto Rios

1. Resumen ejecutivo

La solución implementa un framework de automatización orientado a resolver el reto de modernización de una biblioteca universitaria. El proyecto parte de un modelo PostgreSQL legacy deliberadamente defectuoso, lo puebla con datos sintéticos, valida reglas de calidad declaradas en JSON, transforma la información hacia un modelo relacional normalizado en tercera forma normal y finalmente migra los datos consistentes hacia MongoDB.

El diseño está organizado como una canalización ETL gobernada por un orquestador central en Python. La lógica se divide en módulos especializados para conexión, poblamiento, validación, normalización, migración y utilidades comunes. Esta separación facilita mantenimiento, pruebas, trazabilidad y explicación oral del proyecto.

Desde el punto de vista técnico, las decisiones más importantes fueron: usar PostgreSQL como fuente relacional normalizada, MongoDB como destino documental, JSON como mecanismo externo de reglas y mapeo, Docker Compose para reproducibilidad de infraestructura, Faker para generación de carga y procedimientos almacenados para encapsular reglas de integridad dentro de la base de datos.

2. Artefactos

Artefacto	Rol técnico dentro de la solución
main.py	Orquestador principal del flujo de trabajo: creación de esquema legacy, poblamiento, validación, normalización, objetos de base de datos y migración a MongoDB.
src/db.py	Módulo de conexión a PostgreSQL y MongoDB. Usa context manager para manejar commit, rollback y cierre de conexión.
src/populate_legacy.py	Generador de datos sintéticos con Faker para poblar las tablas legacy con 250 registros por defecto.
src/quality_validator.py	Motor de validación estática de SQL y validación dinámica de datos contra reglas del JSON.
src/normalizer.py	Motor ETL interno que transforma datos sucios hacia el modelo normalizado y usa procedimientos almacenados.
src/migrator.py	Módulo de migración de datos normalizados hacia colecciones MongoDB.
src/utils.py	Utilidades transversales: lectura JSON, configuración de logger y ejecución de archivos SQL.
sql/01_legacy_dirty_schema.sql	Script que reproduce las tablas legacy con problemas de diseño: campos TEXT, datos multivalorados, tipos inconsistentes y ausencia de relaciones.
sql/02_normalized_schema.sql	Script de creación del modelo relacional normalizado en 3FN con PK, FK, restricciones y tablas de auditoría.
sql/03_db_objects.sql	Objetos programables: stored procedures CRUD, vista analítica, función de multa y cursor de auditoría.
config/config_calidad.json	Reglas de validación de calidad para datos crudos y revisión estática de SQL.
config/mapping_mongo.json	Definición declarativa del mapeo entre tablas PostgreSQL normalizadas y colecciones MongoDB.
docker-compose.yml	Infraestructura reproducible para PostgreSQL 16 y MongoDB 7.

tests/test_quality_validator.py	Prueba unitaria básica del validador estático de SQL.
DOCUMENTACION_TECNICA.md	Documentación técnica previa incluida en el proyecto.
README.md	Guía general de propósito, arquitectura y ejecución rápida.

3. Arquitectura general de la solución

La arquitectura se diseñó en capas para separar responsabilidades y evitar un script monolítico. El archivo main.py actúa como capa de orquestación; los módulos de src/ contienen la lógica de negocio; sql/ contiene la definición persistente de estructura y objetos de base de datos; config/ contiene reglas parametrizables; logs/ funciona como salida de trazabilidad.

```

biblioteca_quality_framework/
├── main.py                # Orquestador de flujo completo
├── docker-compose.yml     # PostgreSQL + MongoDB
├── requirements.txt       # Dependencias Python
├── config/
│   ├── config_calidad.json # Reglas de calidad
│   └── mapping_mongo.json  # Mapeo PostgreSQL -> MongoDB
├── sql/
│   ├── 01_legacy_dirty_schema.sql # Esquema legacy sucio
│   ├── 02_normalized_schema.sql   # Esquema normalizado 3FN
│   └── 03_db_objects.sql          # Procedimientos, vista, función y cursor
├── src/
│   ├── db.py
│   ├── populate_legacy.py
│   ├── quality_validator.py
│   ├── normalizer.py
│   ├── migrator.py
│   └── utils.py
├── tests/
└── logs/

```

3.1 Flujo de ejecución

Fase	Qué hace	Componente responsable	Resultado esperado
1	Analiza estáticamente el SQL legacy contra reglas de calidad.	quality_validator.validate_sql_static	Advertencias por uso de TEXT, palabras prohibidas o falta de claves.
2	Crea nuevamente las tablas legacy no normalizadas.	utils.execute_sql_file + 01_legacy_dirty_schema.sql	Base inicial degradada y controlada.
3	Genera datos de prueba masivos usando Faker.	populate_legacy.populate_dirty_tables	Registros sintéticos en Biblioteca_Data, Prestamos_Crudos, Inventario_Sedes y Reseñas_Usuarios.
4	Valida registros contra reglas JSON.	quality_validator.validate_dirty_data	Reporte de total de registros, inválidos y errores.
5	Crea el esquema normalizado y objetos programables.	02_normalized_schema.sql + 03_db_objects.sql	Modelo en 3FN con procedimientos, vista, función y cursor.

6	Transforma datos sucios a tablas normalizadas.	normalizer.normalize_from_dirty	Entidades limpias y relacionadas.
7	Ejecuta ejemplos de vista, función y auditoría.	run_view_and_function_examples	Evidencia de funcionamiento de objetos DB.
8	Migra información a MongoDB.	migrator.migrate_to_mongo	Colecciones books, users, loans e inventory.

4. Decisiones técnicas y justificación

4.1 Python como orquestador

Python fue elegido como capa de automatización porque permite integrar bases de datos relacionales y NoSQL con librerías maduras, ejecutar scripts SQL, procesar datos de forma flexible y generar logs de auditoría. Además, facilita una explicación clara del flujo ETL y permite pruebas unitarias con pytest.

4.2 PostgreSQL como motor relacional

PostgreSQL se usa para representar tanto el modelo legacy como el modelo normalizado. Es adecuado porque soporta claves foráneas, restricciones CHECK, expresiones regulares, procedimientos almacenados PL/pgSQL, vistas, funciones y cursores. Estos elementos eran requerimientos explícitos del reto.

4.3 MongoDB como destino documental

MongoDB se usa para materializar un modelo documental orientado a lectura. La migración construye documentos de libros con autores, categoría y editorial embebidos, evitando consultas relacionales complejas en escenarios donde se consume información del catálogo completo.

4.4 JSON para reglas de calidad y mapeo

Las reglas de negocio no se dejan quemadas en el código. config_calidad.json define obligatoriedad, tipos, longitudes, expresiones regulares, rangos y valores permitidos. mapping_mongo.json documenta el contrato de transformación hacia MongoDB. Esta decisión aumenta mantenibilidad porque cambios de reglas o mapeos no obligan a modificar la lógica principal.

4.5 Docker Compose para infraestructura

docker-compose.yml permite levantar PostgreSQL y MongoDB de forma reproducible. Esto reduce errores de instalación local y garantiza que la sustentación pueda ejecutarse con servicios equivalentes a los esperados.

4.6 Context manager de base de datos

src/db.py implementa pg_connection() con contextmanager. Esta decisión centraliza commit, rollback y cierre de conexión. Si una fase falla, se ejecuta rollback para proteger la consistencia de la base de datos.

4.7 Faker para poblamiento masivo

Faker permite crear datos sintéticos realistas: nombres, correos, direcciones, editoriales y fechas. La generación de 250 registros permite demostrar el estrés del modelo legacy y validar si el proceso de normalización funciona con volumen suficiente.

4.8 Procedimientos almacenados para CRUD

Los stored procedures encapsulan operaciones de inserción, actualización y eliminación de libros. La inserción usa estrategia upsert para crear o reutilizar categorías, editoriales y autores antes de crear el libro, manteniendo la integridad referencial desde la base de datos.

4.9 Vista analítica

La vista vw_libros_mas_prestados consolida información de libros, préstamos y usuarios. La decisión de usar una vista simplifica consultas frecuentes y separa la lógica analítica del código Python.

4.10 Función de multa

fn_calcular_multa centraliza una regla de negocio: calcular una sanción de 1500 COP por día después de 15 días. Mantener esta lógica en la base reduce duplicación si varias aplicaciones consumen la misma regla.

4.11 Cursor de auditoría

sp_auditar_prestamos_activos recorre préstamos activos y registra hallazgos en auditoria_prestamos. El cursor cumple el requerimiento académico y permite explicar procesamiento fila a fila para actividades de auditoría o revisión operacional.

4.12 Idempotencia parcial

El proyecto usa DROP TABLE IF EXISTS en scripts SQL y limpia colecciones MongoDB antes de migrar. Esto permite repetir la ejecución sin duplicar datos en el destino documental y facilita pruebas durante la sustentación.

5. Elaboración del modelo legacy y diagnóstico de calidad

La solución conserva un script legacy equivalente al planteado en el reto para demostrar problemas de diseño. La intención no es corregirlo desde el inicio, sino reproducir el estado defectuoso, poblarlo y luego demostrar la mejora al pasar al modelo normalizado.

Tabla legacy	Problema representado	Impacto técnico
Biblioteca_Data	categoria_y_descripcion combina dos datos en un campo TEXT; fecha_publicacion usa VARCHAR(50).	Viola 1FN, dificulta validación de fecha, búsqueda por categoría y control de longitud.
Prestamos_Crudos	libros_prestados almacena una lista separada por comas; id_prestamo no se define como PK.	Impide integridad referencial y obliga a parsear cadenas para conocer cada préstamo real.
Inventario_Sedes	Repite sede, ubicación y libro asociado sin FK.	Genera redundancia, inconsistencias de ubicación y dificultad para controlar inventario por sede.
Reseñas_Usuarios	calificacion se almacena como VARCHAR(10) y no hay relación con	Permite datos no numéricos y reseñas huérfanas.

	usuarios ni libros.	
--	---------------------	--

5.1 Validación estática del SQL

La función `validate_sql_static` lee el archivo SQL como texto y aplica reglas definidas en `config_calidad.json`. Esta validación permite detectar problemas antes de ejecutar scripts. En el proyecto se configuró la detección de palabras prohibidas como DROP DATABASE o TRUNCATE, el uso de campos TEXT y la exigencia de claves primarias.

```
sql_static_rules:
  forbidden_keywords: ["DROP DATABASE", "TRUNCATE"]
  avoid_text_fields: true
  require_primary_keys: true
  require_foreign_keys_after_normalization: true
```

5.2 Validación dinámica de datos

Después del poblamiento, `validate_dirty_data` consulta cada tabla legacy y valida columna por columna. El reporte resultante incluye total de registros revisados, cantidad de registros inválidos y detalle de errores. Esta separación entre reglas y código es profesional porque el equipo de calidad puede ajustar políticas sin tocar el motor de validación.

Regla	Ejemplo aplicado	Justificación
required	titulo_libro, autor_nombre, correo_usuario	Evita registros incompletos.
max_length	VARCHAR esperados como 255, 1000, etc.	Controla tamaño y evita datos excesivos.
regex	correo_usuario debe cumplir formato con @ y dominio.	Asegura integridad básica de identidad digital.
type	fecha_publicacion como date, calificacion como integer.	Detecta inconsistencias de tipos legacy.
min/max	calificacion entre 1 y 5, cantidad_total >= 0.	Garantiza reglas de dominio.
allowed_values	ACTIVO, DEVUELTO, VENCIDO.	Reduce estados no controlados.
must_contain	categoria_y_descripcion debe contener .	Identifica el patrón legacy que luego será atomizado.
no_empty_items_csv	libros_prestados no debe tener elementos vacíos.	Evita listas multivaloradas defectuosas.

6. Elaboración del modelo normalizado en 3FN

El modelo normalizado se construyó en `sql/02_normalized_schema.sql`. La normalización no solo divide tablas; también incorpora restricciones de integridad, claves primarias, claves foráneas, unicidad y validaciones CHECK. Esto convierte la base en una fuente confiable para la posterior migración NoSQL.

Principio	Aplicación concreta	Resultado
1FN	categoria_y_descripcion se separa en categorias.nombre y categorias.descripcion;	Atributos atómicos y eliminación de listas dentro de columnas.

	libros_prestados se transforma en filas individuales de prestamos.	
2FN	Autores, editoriales y categorías se separan del libro; libros_autores resuelve la relación N:M.	Cada dato depende de la clave de su propia entidad.
3FN	Sedes, inventario, usuarios y auditoría se separan de transacciones y catálogos.	Se eliminan dependencias transitivas y redundancia.

Tabla normalizada	Responsabilidad	Restricciones relevantes
categorias	Catálogo de clasificación bibliográfica.	PK id_categoria, UNIQUE nombre.
editoriales	Catálogo de casas editoriales.	PK id_editorial, UNIQUE nombre.
autores	Catálogo de autores.	PK id_autor, UNIQUE nombre.
libros	Entidad central bibliográfica.	PK id_libro, UNIQUE titulo, FK categoría/editorial, CHECK fecha <= CURRENT_DATE.
libros_autores	Relación muchos a muchos entre libros y autores.	PK compuesta id_libro + id_autor, FK con ON DELETE CASCADE.
usuarios	Usuarios de biblioteca.	PK id_usuario, UNIQUE correo, CHECK regex de correo.
prestamos	Transacciones de préstamo.	FK usuario/libro, CHECK estado, CHECK fechas.
sedes	Ubicaciones físicas.	PK id_sede, UNIQUE nombre.
inventario	Cantidad por sede y libro.	PK compuesta, FK sede/libro, CHECK cantidad >= 0.
resenas	Comentarios y calificación de usuarios.	FK usuario/libro, CHECK calificación 1 a 5, UNIQUE usuario-libro.
auditoria_prestamos	Bitácora de auditoría operacional.	FK préstamo y fecha de auditoría automática.

7. Proceso ETL implementado en Python

7.1 Poblamiento masivo automático

populate_legacy.py usa Faker con locale es_CO por defecto. En cada iteración crea un libro, un préstamo crudo, un registro de inventario y una reseña. Se generan patrones intencionalmente problemáticos: categoría y descripción concatenadas con |, préstamos como CSV y calificación como texto.

Esta fase es importante porque evita entregar un proyecto vacío. Permite sustentar con evidencia que el framework opera sobre datos reales de prueba y no únicamente sobre scripts de estructura.

7.2 Normalización desde datos sucios

normalizer.py realiza la transformación desde las cuatro tablas legacy hacia el modelo 3FN. La función _split_category atomiza la categoría y descripción. Luego se llama al procedimiento sp_insertar_libro para centralizar en la base la creación de libro, autor, categoría y editorial.

En préstamos, la columna libros_prestados se divide por comas y cada elemento genera una fila independiente en prestamos. En inventario, se crea o actualiza la sede y se relaciona con el libro. En reseñas, se resuelve el libro por título y se asocia con un usuario existente.

8. Objetos de base de datos implementados

Objeto	Tipo	Propósito	Justificación técnica
sp_insertar_libro	Stored procedure	Inserta libro y dependencias: categoría, editorial y autor.	Reduce duplicación de lógica y mantiene integridad mediante upsert.
sp_actualizar_libro	Stored procedure	Actualiza título y fecha de publicación por id.	Encapsula operación CRUD de actualización.
sp_eliminar_libro	Stored procedure	Elimina libro por id.	Aprovecha ON DELETE CASCADE en dependencias.
vw_libros_mas_prestados	Vista	Lista libros con total de préstamos y usuarios agregados.	Facilita reporte sin repetir joins en Python.
fn_calcular_multa	Función PL/pgSQL	Calcula multa de mora con regla de 15 días de gracia.	Centraliza regla monetaria reutilizable.
sp_auditar_prestamos_activos	Stored procedure con cursor	Recorre préstamos activos y registra mensajes de auditoría.	Cumple requerimiento de cursores y genera trazabilidad.

9. Migración automática a MongoDB

La migración a MongoDB está implementada en src/migrator.py. El proceso inicia limpiando las colecciones books, users, loans e inventory para que cada ejecución sea reproducible. Luego consulta PostgreSQL y construye documentos JSON compatibles con MongoDB.

La colección books usa un patrón de documento embebido: cada libro incluye category, publisher y authors. Esta decisión es coherente cuando el caso de uso principal es consultar libros con su metadata completa. Para préstamos se conservan referencias user_id y book_id para evitar documentos de usuario demasiado grandes y mantener separación entre transacciones y catálogos.

Colección	Origen relacional	Diseño documental	Motivo
books	libros + categorías + editoriales + autores	Documento con categoría, editorial y autores embebidos.	Optimiza lectura de catálogo completo.
users	usuarios	Documento plano con user_id, name, email.	Consulta simple por identidad.
loans	prestamos	Documento transaccional con referencias a usuario y libro.	Evita duplicar datos de libro/usuario en cada préstamo.
inventory	inventario	Documento con branch_id, book_id y total_quantity.	Representa disponibilidad por sede y libro.

10. Registro, trazabilidad y pruebas

El logger configurado en `src/utlis.py` escribe simultáneamente en consola y en `logs/reporte_calidad.log`. Esta decisión aporta trazabilidad durante la ejecución y deja evidencia para revisar cuántos registros fueron poblados, cuántos fueron inválidos, estadísticas de normalización y resultados de migración.

El proyecto incluye una prueba unitaria en `tests/test_quality_validator.py`. La prueba crea un SQL temporal con un campo TEXT y verifica que `validate_sql_static` lo reporte como hallazgo. Aunque es una prueba mínima, demuestra que la solución fue pensada con posibilidad de validación automatizada.

11. Configuración de entorno

La infraestructura se levanta con Docker Compose y las dependencias Python se instalan desde `requirements.txt`. El proyecto declara `psycopg2-binary` para PostgreSQL, `pymongo` para MongoDB, `Faker` para datos sintéticos, `python-dotenv` para variables de entorno, `jsonschema` como dependencia de validación extensible y `pytest` para pruebas.

Dependencia	Uso dentro del proyecto
<code>psycopg2-binary</code>	Conexión y ejecución SQL sobre PostgreSQL.
<code>pymongo</code>	Conexión e inserción documental en MongoDB.
<code>Faker</code>	Generación de datos sintéticos para pruebas de carga.
<code>python-dotenv</code>	Carga de variables desde <code>.env</code> .
<code>jsonschema</code>	Dependencia disponible para validaciones JSON más formales; el código actual implementa validación propia.
<code>pytest</code>	Ejecución de pruebas unitarias.

12. Trazabilidad contra requerimientos del reto

Requerimiento	Implementación observada	Estado
Archivo <code>main.py</code> orquestador	Existe <code>main.py</code> con flujo completo por fases.	Cumplido
Carpeta <code>sql/</code> con creación, normalización y objetos	Existen <code>01_legacy_dirty_schema.sql</code> , <code>02_normalized_schema.sql</code> y <code>03_db_objects.sql</code> .	Cumplido
<code>config_calidad.json</code>	Existe con reglas estáticas y dinámicas por tabla.	Cumplido
<code>mapping_mongo.json</code>	Existe con definición de colecciones y campos.	Cumplido
Poblamiento de 250 registros	<code>populate_dirty_tables</code> usa <code>total_records=250</code> por defecto desde <code>main.py</code> .	Cumplido
Stored procedures CRUD	<code>sp_insertar_libro</code> , <code>sp_actualizar_libro</code> y <code>sp_eliminar_libro</code> .	Cumplido
Vista de libros más prestados	<code>vw_libros_mas_prestados</code> .	Cumplido
Función de multa	<code>fn_calcular_multa</code> .	Cumplido
Cursor de préstamos activos	<code>sp_auditar_prestamos_activos</code> .	Cumplido
Migración a MongoDB	<code>migrate_to_mongo</code> crea <code>books</code> , <code>users</code> , <code>loans</code> e <code>inventory</code> .	Cumplido
Reporte de calidad	<code>logs/reporte_calidad.log</code> configurado desde logger.	Cumplido

13. Validación técnica

Para validar de manera profesional el funcionamiento de la solución se recomienda ejecutar una secuencia controlada:

```
# 1. Levantar infraestructura
cd biblioteca_quality_framework
docker compose up -d

# 2. Crear entorno Python
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 3. Revisar y completar variables de entorno
cp .env.example .env
# Agregar DATABASE_URL y alinear MONGO_URL/MONGO_URI según el código

# 4. Ejecutar pruebas
pytest -q

# 5. Ejecutar el orquestador
python main.py

# 6. Ver log de calidad
cat logs/reporte_calidad.log

# 7. Validar PostgreSQL
docker exec -it biblioteca_postgres psql -U postgres -d biblioteca_db -c "SELECT COUNT(*) FROM libros;"
docker exec -it biblioteca_postgres psql -U postgres -d biblioteca_db -c "SELECT * FROM vw_libros_mas_prestados LIMIT 5;"

# 8. Validar MongoDB
docker exec -it biblioteca_mongo mongosh biblioteca_mongo --eval "db.books.countDocuments(); db.loans.countDocuments();"

```

14. Conclusiones

La solución está estructurada como un framework profesional y cumple el objetivo principal del reto: demostrar una cadena completa de calidad, automatización, normalización y migración. La separación de componentes, el uso de JSON para reglas, la presencia de objetos programables en PostgreSQL y la migración a MongoDB constituyen decisiones técnicas justificables y alineadas con prácticas empresariales.

El proyecto puede explicarse como una solución ETL gobernada por calidad: primero se mide el problema, luego se normaliza y finalmente se migra a un modelo documental optimizado para consulta.